

Pipeline Templates — DRY Your CI/CD

Author: Rajaneesh K | **Date:** May 2026

50 microservices, 50 pipelines, all doing the same build-test-deploy. Templates let you write it once and reuse everywhere.

The Copy-Paste Problem

Without templates, every new microservice means copying an existing pipeline, tweaking a few values, and hoping you don't introduce a bug. After six months, you have 30 pipelines that were *supposed* to be identical but have drifted apart. One team added a security scan, another forgot to update the agent pool, and nobody can remember which pipeline has the "correct" version.

Templates solve this by extracting common patterns into reusable files. Change the template once → all 50 pipelines get the update. It's the DRY principle applied to CI/CD, and it's the single biggest productivity improvement for teams managing multiple pipelines.

Template Types

Template Type	What It Defines	Include With
Step template	Reusable steps	<code>- template: steps/build.yml</code>
Job template	Reusable jobs	<code>- template: jobs/deploy.yml</code>
Stage template	Reusable stages	<code>- template: stages/release.yml</code>
Variable template	Shared variables	<code>- template: vars/common.yml</code>
Extends template	Enforced wrapper	<code>extends: template: secure.yml</code>

Step Template

templates/steps/docker-build.yml

```
parameters:
  - name: imageName
    type: string
  - name: dockerfilePath
    type: string
    default: 'Dockerfile'
  - name: tag
    type: string
    default: '${Build.BuildId}'

steps:
  - task: Docker@2
    displayName: 'Build ${{ parameters.imageName }}'
    inputs:
      containerRegistry: 'acr-connection'
      repository: '${{ parameters.imageName }}'
      command: 'buildAndPush'
      Dockerfile: '${{ parameters.dockerfilePath }}'
      tags: |
        ${{ parameters.tag }}
        latest
```

Usage in main pipeline

```
# azure-pipelines.yml
steps:
  - template: templates/steps/docker-build.yml
    parameters:
      imageName: 'my-api'
      tag: 'v1.2.3'

  - template: templates/steps/docker-build.yml
    parameters:
      imageName: 'my-worker'
      dockerfilePath: 'worker/Dockerfile'
```

Job Template

templates/jobs/deploy-to-aks.yml

```
parameters:
  - name: environment
    type: string
  - name: namespace
    type: string
  - name: aksServiceConnection
    type: string
  - name: imageTag
    type: string

jobs:
  - deployment: Deploy_${{ parameters.environment }}
    displayName: 'Deploy to ${{ parameters.environment }}'
    pool:
      vmImage: 'ubuntu-latest'
    environment: '${{ parameters.environment }}'
    strategy:
      runOnce:
        deploy:
          steps:
            - download: current
              artifact: manifests

            - task: KubernetesManifest@1
              inputs:
                action: 'deploy'
                kubernetesServiceConnection: '${{ parameters.aksServiceConnection }}'
                namespace: '${{ parameters.namespace }}'
                manifests: '$(Pipeline.Workspace)/manifests/*.yaml'
                containers: 'myacr.azurecr.io/myapp:${{ parameters.imageTag }}'
```

Usage — Deploy to 3 environments with same template

```
stages:
- stage: DeployDev
  jobs:
  - template: templates/jobs/deploy-to-aks.yml
    parameters:
      environment: 'dev'
      namespace: 'dev'
      aksServiceConnection: 'aks-dev'
      imageTag: '$(Build.BuildId)'

- stage: DeployStaging
  jobs:
  - template: templates/jobs/deploy-to-aks.yml
    parameters:
      environment: 'staging'
      namespace: 'staging'
      aksServiceConnection: 'aks-staging'
      imageTag: '$(Build.BuildId)'

- stage: DeployProd
  jobs:
  - template: templates/jobs/deploy-to-aks.yml
    parameters:
      environment: 'production'
      namespace: 'prod'
      aksServiceConnection: 'aks-prod'
      imageTag: '$(Build.BuildId)'
```

Stage Template

templates/stages/full-release.yml

```
parameters:
  - name: environments
    type: object
    default:
      - name: dev
        namespace: dev
        connection: aks-dev
      - name: staging
        namespace: staging
        connection: aks-staging
      - name: production
        namespace: prod
        connection: aks-prod

stages:
  - ${ each env in parameters.environments }:
    - stage: Deploy_${ env.name }
      displayName: 'Deploy to ${ env.name }'
      jobs:
        - template: ../jobs/deploy-to-aks.yml
          parameters:
            environment: '${ env.name }'
            namespace: '${ env.namespace }'
            aksServiceConnection: '${ env.connection }'
            imageTag: '${Build.BuildId}'
```

Variable Template

templates/vars/common.yml

```
variables:
  containerRegistry: 'myacr.azurecr.io'
  buildConfiguration: 'Release'
  dotnetVersion: '8.0'
  nodeVersion: '18'
```

Usage

```
variables:
  - template: templates/vars/common.yml
  - name: imageName
    value: 'my-service'
```

Cross-Repo Templates

```
# Reference templates from another repository
resources:
  repositories:
    - repository: templates
      type: git
      name: MyProject/pipeline-templates # Another repo in same org
      ref: refs/heads/main

# Use template from that repo
stages:
  - template: stages/dotnet-ci.yml@templates
    parameters:
      solution: '**/*.sln'

  - template: stages/aks-deploy.yml@templates
    parameters:
      environment: 'production'
```

Best Practice: Keep templates in a dedicated "pipeline-templates" repo. All microservice repos reference it. Update templates once → all pipelines get the update.

Extends Template (Security Enforcement)

The `extends` keyword is the most powerful template feature — it wraps your entire pipeline and enforces organizational policies. If the security team mandates container scanning before every deployment, `extends` ensures no team can skip it, even accidentally. The central platform team owns the template, and individual teams plug their stages into it.

```
# templates/secure-pipeline.yml – Enforced by org
parameters:
  - name: stages
    type: stageList

stages:
  - stage: SecurityScan
    jobs:
      - job: Scan
        steps:
          - script: trivy image $(imageName)
            displayName: 'Container security scan'

  - ${ parameters.stages } # Team's stages go here

  - stage: Compliance
    jobs:
      - job: AuditLog
        steps:
          - script: echo "Deployment logged for compliance"
```

```
# Team's pipeline – MUST use extends
extends:
  template: templates/secure-pipeline.yml@templates
parameters:
  stages:
    - stage: Build
      jobs:
        - job: BuildApp
          steps:
            - script: dotnet build
```

Template Parameter Types

Azure Pipelines templates support several parameter types for validation and type safety. Using typed parameters catches errors at compile time, before the pipeline even runs.

Type	Description	Example
string	Text value	imageName: 'myapp'
boolean	true/false	runTests: true
number	Integer value	replicas: 3
object	Complex data structure	Lists, maps, nested values
stepList	List of steps	Inject steps into template
jobList	List of jobs	Inject jobs into template
stageList	List of stages	Used with extends

```
# Template with typed parameters + defaults + allowed values
parameters:
  - name: environment
    type: string
    values: ['dev', 'staging', 'production'] # Enum-like validation
  - name: runTests
    type: boolean
    default: true
  - name: replicas
    type: number
    default: 2
  - name: extraSteps
    type: stepList
    default: []

steps:
  - script: echo "Deploying to ${ parameters.environment }"
  - ${ if parameters.runTests }:
    - script: npm test
  - ${ parameters.extraSteps }
```

Recommended Folder Structure

```
pipeline-templates/          # Shared repo
├─ steps/
│   ├── docker-build.yml
│   ├── dotnet-build.yml
│   ├── npm-build.yml
│   └─ run-tests.yml
├─ jobs/
│   ├── deploy-to-aks.yml
│   ├── deploy-to-appservice.yml
│   └─ run-integration-tests.yml
├─ stages/
│   ├── dotnet-ci-cd.yml
│   ├── node-ci-cd.yml
│   └─ full-release.yml
└─ vars/
    ├── common.yml
    ├── dev.yml
    ├── staging.yml
    └─ prod.yml
```

Takeaway: Templates eliminate copy-paste across pipelines. Start with step templates (build, test, scan), graduate to job templates (deploy), then stage templates (full CI/CD flows). Use a shared "pipeline-templates" repo so all services reference the same patterns. Use extends templates to enforce security policies org-wide. If you have more than 3 pipelines doing the same thing — you need templates.

Tomorrow: Pipeline Variables & Variable Groups — Managing Secrets.

#DevOps #AzureDevOps #CICD #PipelineTemplates #DRY #100DaysOfDevOps #Day31